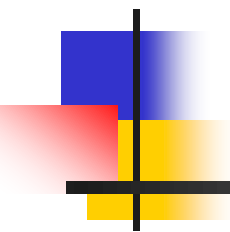


DATA STRUCTURES AND ALGORITHMS



Lecture Notes 4

Pradit Pitaksathienkul



ROAD MAP

- Abstract Data Types (ADT)
 - The List ADT
 - List implementation of lists
 - Linked list implementation of lists
 - **The Stack ADT**
 - **Linked list implementation of stacks**
 - List implementation of stacks
 - The Queue ADT
 - Link list implementation of queues
 - List implementation of queues



THE STACK ADT

- A stack is a list
 - Insertions and deletions from one end (top)
 - LIFO (Last In First Out)

Operations:

Push

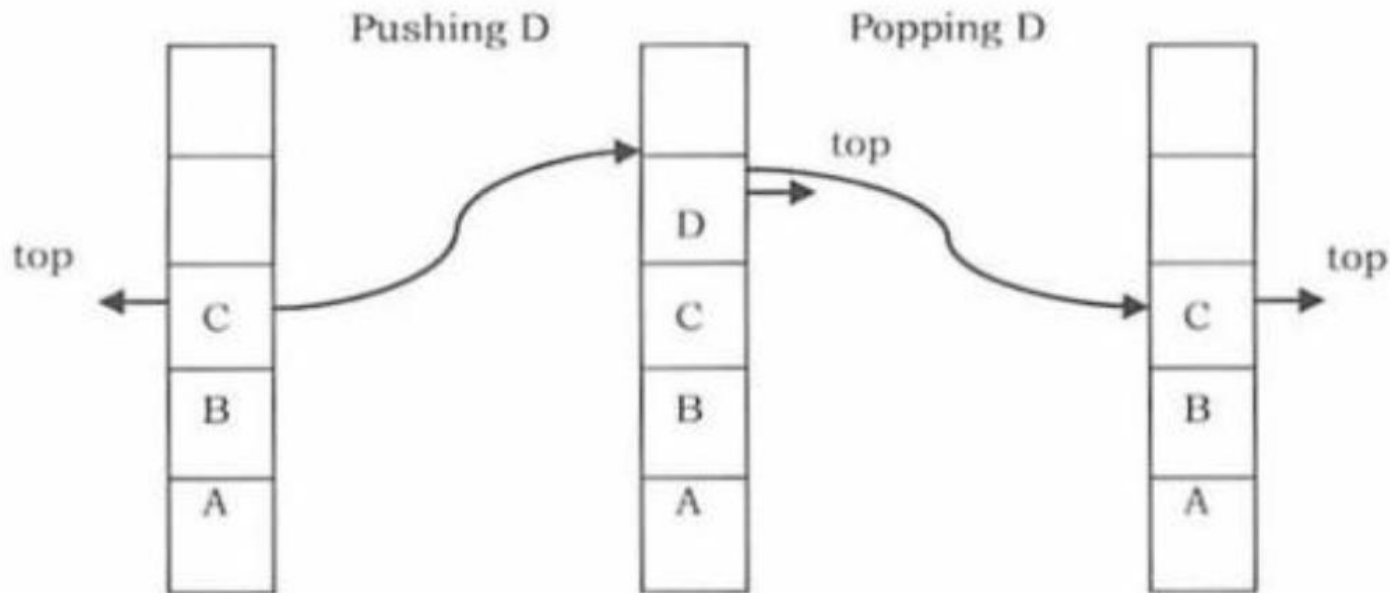
Pop

Top

isEmpty

isFull

THE STACK ADT



Only the top element is accessible !



Implementation of Stacks

1. Linked List Implementation

- insert / delete at the end of the list
 - performs “push” by inserting
 - performs “pop” by deleting
- all operations take constant time



ROAD MAP

- Abstract Data Types (ADT)
 - The List ADT
 - List implementation of lists
 - Linked list implementation of lists
 - The Stack ADT
 - Linked list implementation of stacks
 - **List implementation of stacks**
 - The Queue ADT
 - Link list implementation of queues
 - List implementation of queues



Implementation of Stacks

2. List Implementation

- more popular solution
- need to declare the size
 - usually the max stack size is known
- keep a list of Stack (in Python)



List Implementation of Stacks

- Push → use method `append()`
- Pop → use method `pop()`
- Top → find how many data in stack(`len`)
return data at position(`len-1`)


```
class Stack:
    def __init__(self, limit = 10):
        self.items = []
        self.limit = limit
    def is_empty(self):
        return len(self.items) <= 0
    def push(self, item):
        self.items.append(item)
    def pop(self):
        return self.items.pop()
    def top(self):
        return self.items[len(self.items)-1]
    def size(self):
        return len(self.items)
    def prinstack(self):
        for i in range(len(self.items)):
            print(self.items[i], end=' ')
```

For example: at main program

```
s = stack(5)
```

```
s.push(1)
```

```
s.push(2)
```

```
s.push(3)
```

```
s.push(4)
```

```
s.push(5)
```

```
print('Stack has data: ', end=' ')
```

```
s.prinstack()
```

```
print()
```

```
print('Stack has size: ', s.size())
```

Result:

```
Stack has data:  1 2 3 4 5
```

```
                [0][1]  [4]
```

```
Stack has size:  5
```

```
For example: at main program (continue)
s.pop()
s.pop()
print('Stack has data: ', end=' ')
s.prinstack()
print()
print('Stack has size: ', s.size())
```

Result:

Stack has data: 1 2 3

Stack has size: 3



Special cases of Stacks

1. Stack is full, when push
2. Stack is empty, when pop or top



Special cases of Stacks

1. Stack is full, when push:
We check the size of stack before push.

```
def push(self, item):  
    if len(self.items) >= self.limit:  
        print ('Stack Overflow, Cannot push ',item)  
    else:  
        self.items.append(item)
```



Special cases of Stacks

2. Stack is empty, when pop or top:
We check the size of stack before pop.

```
def pop(self):  
    if len(self.items) <= 0:  
        print ('Stack Underflow!')  
    else:  
        return self.items.pop()
```



Note:

```
def push(self, item):
    if len(self.items) >= self.limit:
        print ('Stack Overflow, Cannot push ', item)
    else:
        self.items.append(item)
def pop(self):
    if len(self.items) <= 0:
        print ('Stack Underflow!')
    else:
        return self.items.pop()
```

Method `append()` and `pop()` is method of data type list of python.



Applications

PostFix Expressions

$5+4*2$

$(5+4)*2$



infix expressions

paranthesis is necessary

- What if you have a simple calculator ?

$5\ 4\ +\ 2\ *$

$5\ 4\ 2\ *\ +$

- How to evaluate postfix expressions ?

- use a stack

- a number is seen → push onto stack

- an operator is seen → pop two values from stack

apply operation

push the result onto stack

Applications

PostFix Expressions

■ Example :

6 5 + 8 *

